Nama : Eka Ahmad Ferdinand Mahtrus

PROGRESS I

KONTROL MOTOR DASAR

1.1. Tujuan Modul

- Memahami cara kerja dan pengontrolan Motor DC menggunakan mikrokontroler ESP32.
- Menerapkan konsep PWM (Pulse Width Modulation) untuk mengatur kecepatan putaran motor.
- Mengintegrasikan komunikasi Serial untuk mengirimkan perintah kontrol secara real-time dari komputer ke mikrokontroler.
- Membuat sistem indikator status operasi motor menggunakan LED.

1.2. Skema Wiring (Rangkaian)

Sistem ini dirancang untuk dihubungkan dengan driver motor DC (seperti modul L298N atau L293D). Berikut adalah panduan penyambungan pin:

- ESP32 ke Driver Motor:
    - Pin 27 dihubungkan ke IN1 (Mengontrol arah putaran A).
    - Pin 26 dihubungkan ke IN2 (Mengontrol arah putaran B).
    - Pin 12 dihubungkan ke ENA (Mengontrol kecepatan/PWM).
- ESP32 Indikator:
    - Pin 2 terhubung ke LED internal ESP32 (sebagai indikator motor menyala/berputar).
- Power Supply:
    - Pastikan driver motor mendapatkan suplai daya eksternal yang sesuai (misal baterai 9V/12V), dan *ground* (GND) antara ESP32 dan Driver Motor saling terhubung (Common Ground).

1.3. Konsep & Logika Sistem

Sistem beroperasi dengan membaca input berupa karakter atau teks melalui *Serial Monitor*. Setelah input diterima, program akan mengidentifikasi perintah tersebut dan mengubah nilai variabel dutyCycle.

Nilai dutyCycle ini (berkisar antara 0 - 255) kemudian dikonversi menjadi sinyal PWM yang dikirimkan ke pin *Enable* (ENA) pada driver motor. Konfigurasi pin IN1 (HIGH) dan IN2 (LOW) dikunci agar motor berputar pada satu arah secara konsisten. Jika dutyCycle lebih besar dari 0, LED indikator akan menyala, menandakan motor sedang aktif.

1.4. Daftar Perintah pada Program

- 1 : **Motor ON**. Menyalakan motor dengan kecepatan default (atau kecepatan terakhir sebelum dimatikan).
- 0 : **Motor OFF**. Mematikan motor secara paksa (mengatur duty cycle ke 0).
- + : **Naikkan Kecepatan**. Menambah nilai duty cycle sebesar 10 poin (Maksimal 255).
- - : **Turunkan Kecepatan**. Mengurangi nilai duty cycle sebesar 10 poin (Minimal 0).
- PWM:xxx : **Set Manual**. Mengatur kecepatan motor secara spesifik. Ganti xxx dengan angka 0 hingga 255 (Contoh: PWM:150).

1.5. Source Code & Penjelasan Logika Program

```
//PROGRESS 1 KONTROL MOTOR DASAR / CONNECT WITH WIRE
// ============================================================
// Kontrol Motor DC + LED dengan PWM via Serial
// Perintah:
//   '1'      -> Motor ON (duty terakhir / default)
//   '0'      -> Motor OFF
//   '+'      -> Naikkan kecepatan (+10)
```

```
//   '-'       -> Turunkan kecepatan (-10)

//   'PWM:xxx'  -> Set duty cycle langsung (0-255)

// ============================================================




// --- Konfigurasi Pin Hardware ---

const int ledPin = 2;       // Pin untuk LED indikator (menggunakan LED built-in ESP32)

const int motor1Pin1 = 27;  // Pin IN1 pada driver motor (mengatur arah putaran)

const int motor1Pin2 = 26;  // Pin IN2 pada driver motor (mengatur arah putaran)

const int enable1Pin = 12;  // Pin ENA pada driver motor (mengatur kecepatan via PWM)



// --- Konfigurasi PWM ESP32 (ledc) ---

const int freq = 30000;       // Frekuensi PWM (30 kHz cocok untuk motor DC)

const int pwmChannel = 0;     // Channel PWM ESP32 (0-15)

const int resolution = 8;     // Resolusi 8-bit (nilai 0 - 255)



// --- Variabel Kontrol Kecepatan ---

int dutyCycle = 200;          // Nilai kecepatan awal (default) saat motor dinyalakan

const int DUTY_STEP = 10;     // Besar penambahan/pengurangan kecepatan tiap perintah '+' atau '-'

const int DUTY_MIN  = 0;      // Batas minimum kecepatan (Motor mati)

const int DUTY_MAX  = 255;    // Batas maksimum kecepatan (Kecepatan penuh)



// --- Variabel Buffer Serial ---
```

```
String inputString = "";      // Menyimpan string karakter yang diterima dari Serial Monitor


void setup() {

  // Inisialisasi pin sebagai OUTPUT

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);

  pinMode(motor1Pin2, OUTPUT);

  pinMode(enable1Pin, OUTPUT);


  // Inisialisasi komunikasi Serial pada baudrate 115200

  Serial.begin(115200);


  // Konfigurasi modul hardware PWM pada ESP32

  ledcSetup(pwmChannel, freq, resolution);


  // Hubungkan channel PWM ke pin Enable motor

  ledcAttachPin(enable1Pin, pwmChannel);


  // Set kondisi awal: Motor berhenti (Brake/Coast)

  digitalWrite(motor1Pin1, HIGH);

  digitalWrite(motor1Pin2, LOW); // Kombinasi HIGH-LOW membuat motor berputar searah jarum jam

  ledcWrite(pwmChannel, 0);      // Kecepatan 0 (mati)


  // Pesan sambutan di Serial Monitor
```

```
  Serial.println("========================================");

  Serial.println(" ESP32 Motor Controller Siap Digunakan!  ");

  Serial.println("========================================");

  Serial.println("Perintah: 1=ON, 0=OFF, +=Naik, -=Turun, PWM:xxx=Set Manual");

}



/*

 * Fungsi untuk menerapkan nilai dutyCycle saat ini ke hardware.

 * Dipanggil setiap kali ada perubahan kecepatan.

 */

void applyMotor() {

  // Pastikan arah putaran motor konsisten

  digitalWrite(motor1Pin1, HIGH);

  digitalWrite(motor1Pin2, LOW);



  // Terapkan sinyal PWM ke driver motor

  ledcWrite(pwmChannel, dutyCycle);



  // Nyalakan LED jika motor bergerak (dutyCycle > 0), matikan jika berhenti

  digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW);

}



void MotorOn() {

  // Jika kecepatan sebelumnya 0, kembalikan ke default (200) agar motor bisa berputar
```

```
  if (dutyCycle == 0) dutyCycle = 200;
  applyMotor();
  Serial.print("[INFO] Motor ON, duty cycle = ");
  Serial.println(dutyCycle);
}

void MotorOff() {
  dutyCycle = 0;
  applyMotor();
  Serial.println("[INFO] Motor OFF, duty cycle = 0");
}

void naikkanKecepatan() {
  dutyCycle += DUTY_STEP;
  if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX; // Mencegah nilai
melebihi 255
  applyMotor();
  Serial.print("[INFO] Kecepatan NAIK, duty cycle = ");
  Serial.println(dutyCycle);
}

void turunkanKecepatan() {
  dutyCycle -= DUTY_STEP;
  if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN; // Mencegah nilai negatif
  applyMotor();
```

```
  Serial.print("[INFO] Kecepatan TURUN, duty cycle = ");
  Serial.println(dutyCycle);
}

/*
 * Fungsi untuk memproses perintah set manual (PWM:xxx)
 */
void setDuty(int val) {
  // Validasi nilai input agar tetap berada di rentang 0-255
  if (val < DUTY_MIN) val = DUTY_MIN;
  if (val > DUTY_MAX) val = DUTY_MAX;

  dutyCycle = val;
  applyMotor();
  Serial.print("[INFO] Set duty cycle manual = ");
  Serial.println(dutyCycle);
}

void loop() {
  // Selama ada data yang masuk dari Serial Monitor
  while (Serial.available() > 0) {
    char c = Serial.read();

    // Jika mendeteksi karakter Enter/Newline (Tanda perintah selesai diketik)
    if (c == '\n' || c == '\r') {
```

```
inputString.trim(); // Hapus spasi ekstra di awal dan akhir string


// 1. Cek apakah perintah adalah format "PWM:xxx"
if (inputString.startsWith("PWM:")) {
  int val = inputString.substring(4).toInt(); // Ambil angka setelah "PWM:"
  setDuty(val);
}
// 2. Cek apakah ada perintah satu karakter
else if (inputString.length() > 0) {
  char cmd = inputString.charAt(0);


  if (cmd == '1') {
    MotorOn();
  }
  else if (cmd == '0') {
    MotorOff();
  }
  else if (cmd == '+') {
    naikkanKecepatan();
  }
  else if (cmd == '-') {
    turunkanKecepatan();
  }
  else {
    Serial.print("[ERROR] Perintah tidak dikenal: ");
```

```
        Serial.println(inputString);

      }

    }



    // Kosongkan buffer setelah perintah dieksekusi agar siap untuk perintah selanjutnya

    inputString = "";

  }

  else {

    // Tambahkan karakter yang masuk ke dalam buffer

    inputString += c;

  }

 }

}
```

- **Penjelasan Program**

Secara arsitektur, program diawali dengan deklarasi pin kendali motor yang memanfaatkan topologi *H-Bridge* pada modul *driver* (seperti L298N atau L293D). Pin 27 (IN1) dan Pin 26 (IN2) digunakan untuk menetapkan arah putaran motor yang dipatenkan bergerak maju melalui kombinasi logika HIGH dan LOW, sedangkan Pin 12 (ENA) difungsikan sebagai penerima sinyal *Pulse Width Modulation* (PWM) untuk mengatur kecepatan. Selain itu, program juga menginisialisasi Pin 2 sebagai luaran indikator visual (LED).

Sinyal PWM dibangkitkan menggunakan fungsi bawaan ESP32 (ledcSetup dan ledcAttachPin) dengan menetapkan parameter frekuensi sebesar 30.000 Hz dan resolusi 8-bit. Pemilihan frekuensi tinggi (30 kHz) ini dilakukan secara spesifik agar gelombang beroperasi di atas ambang batas pendengaran manusia (20 kHz), sehingga secara efektif mengeliminasi suara dengungan (*whining noise*) pada kumparan motor saat beroperasi pada kecepatan rendah. Di sisi lain, resolusi 8-bit memetakan tingkat kecepatan motor

ke dalam rentang rasio tugas (*duty cycle*) dengan nilai numerik dari 0 (berhenti) hingga 255 (kecepatan maksimal).

Struktur kendali logik dipisahkan ke dalam beberapa fungsi modular yang spesifik (MotorOn, MotorOff, naikkanKecepatan, turunkanKecepatan, dan setDuty). Seluruh fungsi modifikasi kecepatan tersebut bermuara pada pemanggilan fungsi sentral applyMotor(). Fungsi sentral ini mengeksekusi pembaruan nilai PWM ke mikrokontroler sekaligus memuat logika indikator *ternary* (dutyCycle > 0 ? HIGH : LOW) yang memastikan LED hanya menyala apabila motor benar-benar memiliki nilai *duty cycle* di atas nol.

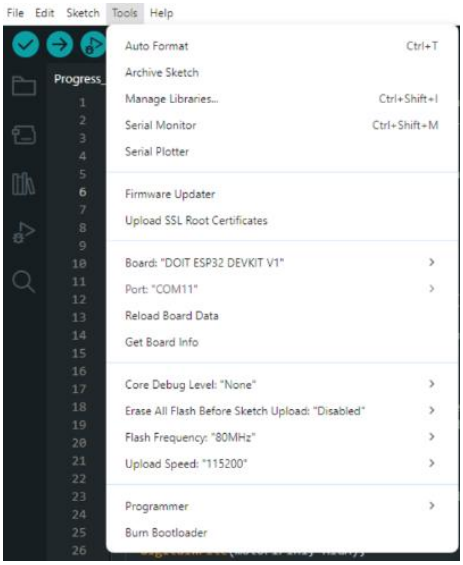
Selain itu, program mengimplementasikan mekanisme pengungkit torsi awal (*starting torque*). Ketika pengguna menghidupkan motor dari keadaan diam murni, mikrokontroler tidak memulainya dari nilai PWM yang terlalu rendah agar terhindar dari kondisi *stall*. Sistem secara otomatis memberikan lonjakan tegangan awal bernilai PWM 200 (if (dutyCycle == 0) dutyCycle = 200;) untuk memastikan motor memiliki daya yang cukup untuk mengatasi inersia mekanis.

Pada sisi antarmuka, keseluruhan sistem diorkestrasi melalui komunikasi Serial. Program memanfaatkan perulangan while (Serial.available() > 0) dan mekanisme *buffering* pada variabel String untuk membaca masukan dari pengguna secara *real-time*. Logika pembacaan ini dirancang untuk mendeteksi karakter *newline* atau *carriage return* sebagai penanda akhir perintah, yang kemudian diproses menjadi dua cabang utama: identifikasi perintah karakter tunggal untuk operasional dasar, atau pemotongan teks (*substring*) untuk mengekstrak dan mengonversi perintah setelan parameter presisi (contoh: "PWM:150") ke dalam bentuk *integer*.

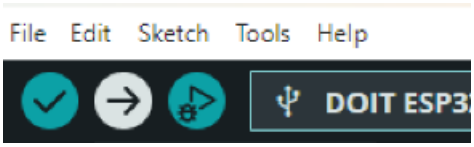
1.6. Hasil Pengujian & Analisis

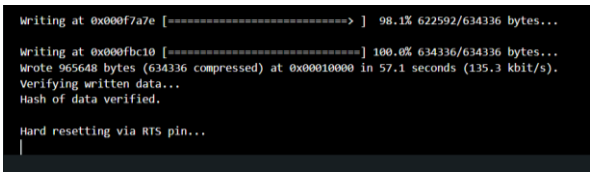
- **Proses Kompilasi dan Upload Program**

Sebelum melakukan komplasi dan upload program, pastikan terlebih dahulu bahwa port komunikasi sudah terhubung dengan benar ke kit ESP32. Pastikan jenis board yang dipilih adalah DOIT ESP32 DEVKIT V1 dengan kecepatan upload (upload speed) sebesar 115200.



Gambar 1. Menu Tools Pemilihan Board dan Port Arduino IDE

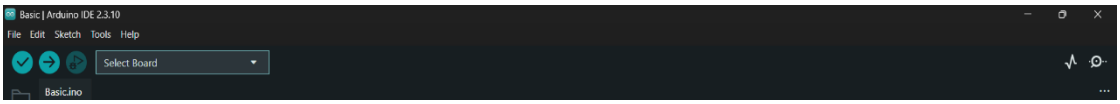




Gambar 2 & 3. Proses Upload dan Kompilasi

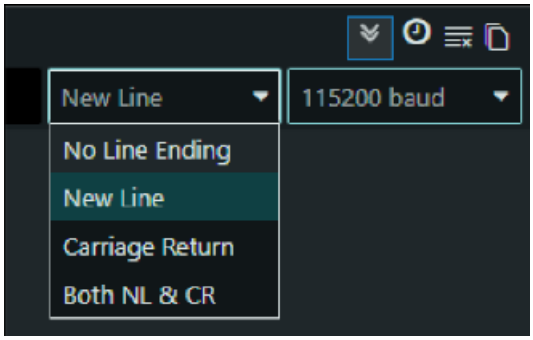
Jika semua langkah pada Gambar 1 sudah diterapkan, lalu selanjutnya adalah menekan tombol upload dan menanamkan kode ke dalam mikrokotroler. Dan tunggu sampai proses upload selesai. Jika code berhasil di unggah tanpa eror, maka pada panel Output akan muncul log keterangan ” Hard resetting via RTS PIN”

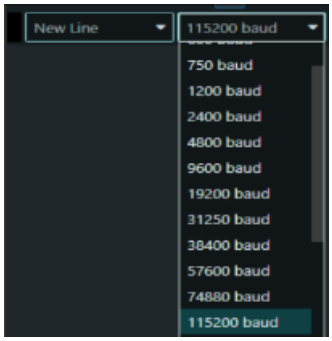
- **Proses Pengunjian Via Serial Monitor**



Gambar 4. Icon Serial Monitor

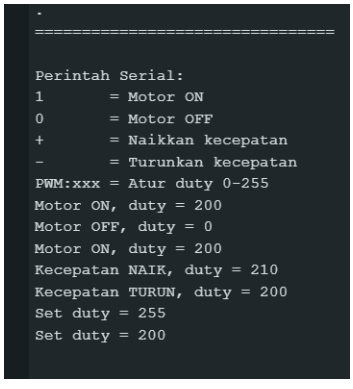
Lalu setelah program berhasil ditanamkan pengujian dapat dilakukan melalui antarmuka Serial Monitor. Dengan mengklik ikon pada pojok kanan atas ”Serial Monitor” maka feature tersebut akan terbuka. Fungsi dari Serial Monitor ini adalah untuk memberikan perinta teks (input) ke perangkat ESP32 sekaligus memantau status mikrokontroler secara real-time.





Gambar 5 & 6. Pengaturan Opsi Line Ending dan Baudrate.

Sebelum melakukan uji coba pengiriman perintah ke motor, terdapat dua parameter krusial yang harus disesuaikan di pojok kanan jendela Serial Monitor. Pastikan pengaturan format baris (*Line Ending*) berada pada opsi New Line dan pengaturan kecepatan komunikasi (*Baudrate*) berada di angka 115200 baud agar selaras dengan kodingan Serial.begin(115200).



Gambar 8. Log hasil pengujian

Jika konfigurasi pada gambar 5 & 6 sudah sesuai, pengujian dapat langsung di lakukan sesuai dengan perintah serial pada Program ini.

- **Analisa Keseluruhan**

Berdasarkan log hasil pengujian yang ditunjukkan pada Gambar 8, secara keseluruhan rancang bangun sistem kendali motor DC berbasis ESP32 telah beroperasi dengan sangat baik dan sesuai dengan parameter perancangan. Pengujian memvalidasi bahwa mikrokontroler mampu menerima, memproses, dan menerjemahkan masukan karakter dari antarmuka *Serial Monitor* menjadi instruksi kendali perangkat keras secara presisi dan seketika (*real-time*). Sinkronisasi komunikasi serial pada kecepatan 115200 *baud* terbukti stabil tanpa adanya indikasi kehilangan data (*data loss*) atau penundaan (*delay*) yang berarti.

Sistem juga menunjukkan tingkat keandalan yang tinggi dalam mengorkestrasi variasi perintah logika. Perintah operasional biner seperti '1' (ON) dan '0' (OFF) berhasil dieksekusi untuk mengubah status aktuasi motor secara instan. Lebih lanjut, algoritma manipulasi rasio tugas (*duty cycle*) terbukti terstruktur dengan baik; fungsi penyesuaian kecepatan bertahap menggunakan parameter '+' dan '-' serta fungsi penetapan nilai mutlak melalui format "PWM:xxx" berhasil memodifikasi kecepatan motor dalam batas aman resolusi 8-bit (0 hingga 255).

Secara perangkat keras, integrasi sinyal *Pulse Width Modulation* (PWM) berfrekuensi 30 kHz ke pin *Enable* (ENA) pada *driver* motor sukses mengatur torsi dan putaran motor secara halus, didukung dengan respons indikator LED yang beroperasi selaras dengan pergerakan motor. Konfigurasi ini menjadikan sistem sangat stabil, modular, dan siap untuk diintegrasikan lebih lanjut ke dalam purwarupa robotika atau sistem otomasi yang lebih kompleks.